Zhou SC, Wang YZ, Wen H *et al.* Balanced quantization: An effective and efficient approach to quantized neural networks. JOURNAL OF COMPUTER SCIENCE AND TECHNOLOGY 32(4): 667–682 July 2017. DOI 10.1007/s11390-017-1750-y

Balanced Quantization: An Effective and Efficient Approach to Quantized Neural Networks

Shu-Chang Zhou [1,2,3], Yu-Zhi Wang [3,4], *Student Member, IEEE*, He Wen [3,5], Qin-Yao He [3,5], and Yu-Heng Zou [5,6]

[1] *University of Chinese Academy of Sciences, Beijing 100049, China*

[2] *State Key Laboratory of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences Beijing 100190, China*

[3] *Megvii Inc., Beijing 100190, China*

[4] *Department of Electronic Engineering, Tsinghua University, Beijing 100084, China*

[5] *Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China*

[6] *School of Electronics Engineering and Computer Science, Peking University, Beijing 100871, China*

E-mail: shuchang.zhou@gmail.com; yz-wang12@mails.tsinghua.edu.cn; {wenhe, hqy, zouyuheng}@megvii.com

Received December 20, 2016; revised May 18, 2017.

Abstract Quantized neural networks (QNNs), which use low bitwidth numbers for representing parameters and performing computations, have been proposed to reduce the computation complexity, storage size and memory usage. In QNNs, parameters and activations are uniformly quantized, such that the multiplications and additions can be accelerated by bitwise operations. However, distributions of parameters in neural networks are often imbalanced, such that the uniform quantization determined from extremal values may underutilize available bitwidth. In this paper, we propose a novel quantization method that can ensure the balance of distributions of quantized values. Our method first recursively partitions the parameters by percentiles into balanced bins, and then applies uniform quantization. We also introduce computationally cheaper approximations of percentiles to reduce the computation overhead introduced. Overall, our method improves the prediction accuracies of QNNs without introducing extra computation during inference, has negligible impact on training speed, and is applicable to both convolutional neural networks and recurrent neural networks. Experiments on standard datasets including ImageNet and Penn Treebank confirm the effectiveness of our method. On ImageNet, the top-5 error rate of our 4-bit quantized GoogLeNet model is 12.7%, which is superior to the state-of-the-arts of QNNs.

Keywords quantized neural network, percentile, histogram equalization, uniform quantization

1 Introduction

Deep neural networks (DNNs) have attracted considerable research interests over the past decade. In various applications, including computer vision[1-4], speech recognition[5-6], natural language processing[7-9], and computer games[10-11], DNNs have demonstrated their ability to model nonlinear relationships in massive amount of data and their robustness to real-world noise. However, the modeling capacities of DNNs are roughly proportional to their computational complexity and number of parameters[12]. Hence many DNNs, like VGGNet[13], GoogLeNet[14] and ResNet[15], which are widely used in computer vision applications, require billions of multiply-accumulate operations (MACs) even for an input image of width and height of 224. Moreover, as these DNN models use many channels of activations (feature maps) for intermediate representations, they have a large runtime memory footprint and storage size. Such vast amount of resource requirement impedes the adoption of DNNs on devices with limited computation resource and power supply[16], and in user-interactive scenarios where instant responses are expected. A similar argument also applies to recur-

Regular Paper
Special Issue on Deep Learning
©2017 Springer Science + Business Media, LLC & Science Press, China

rent neural networks (RNNs). In particular, the transition and embedding matrices in a long short time memory (LSTM)[17] or a gated recurrent unit (GRU)[18] model have dense connections that make them particularly demanding in both computation and storage.

Many approaches have been proposed to accelerate the computation or reduce the memory footprint and storage size of DNNs. One approach from the hardware perspective is designing hardware accelerators for the computationally expensive operations in DNNs[19-21]. From the algorithmic perspective, a popular route to faster and smaller models is to impose constraints on the parameters of a DNN to reduce the number of free parameters and computational complexity, like low-rankness[22-27], sparsity[28-31], circulant property[32], and sharing of weights[33-34]. However, these methods use high bitwidth numbers for computations in general, which requires the availability of high precision MAC instructions that incur high hardware complexity[35]. In contrast, several previous studies have demonstrated that low bitwidth numbers may be sufficient for performing inferences with DNNs. For example, in [36-38], trained DNNs are quantized to use 8-bit numbers for storing parameters and performing computations, without incurring significant degradation of prediction quality. Gong *et al.*[39] also applied vector quantization to speed up inferences of DNNs. However, these studies[36-39] do not integrate the quantization operations into the training process of a DNN, as the discrete quantized values necessarily would have zero gradients, which would break the Back-Propagation (BP) algorithm. Applying quantization as a post-processing step is far from satisfactory as the quantized DNNs do not have a chance to adapt to the quantization errors[40]. Consequently, 8-bit was generally taken to be a limit for post-training quantization of DNNs[41].

Recently, quantized neural networks (QNNs)[42-46] have been proposed to further reduce the bitwidths of DNNs, by incorporating quantization into the training process. The key enabling technique is a trick called straight through estimator (STE)[47-49], which is based on the following observation: as the quantized value is an approximation of the original value, we can substitute the gradient with respect to the quantized value for the gradient of original value. Simple as it is, the trick allows the inclusion of quantization into the computation graph of BP and allows QNNs to represent parameters, activations and gradients with low bitwidth numbers. The QNN technique has been successfully applied to both CNNs and RNNs[50-51], to successfully produce lower bitwidth versions of AlexNet, ResNet-18 and GoogLeNet that have comparable prediction accuracies as their floating point counterparts. However, the degradation of prediction accuracy is still significant for most QNNs, especially when quantizing to less than 4-bit[52-53].

In this paper, we propose a balanced quantization method that improves the prediction accuracies of QNNs. In general, QNNs employ uniform quantization to eliminate floating point operations during the inference process by exploiting the bitwise operations. However, the parameters of neural networks often have a bell-shaped distribution and sporadic large outliers, making the quantized values not evenly distributed among possible values when uniform quantization is applied. In the extreme case, some of the possible quantized values are never used. To remedy this, we propose to use a novel quantization method that ensures the balanced distribution of quantized values.

This paper makes the following contributions.

1) We propose a balanced quantization method for the quantization of parameters of QNNs. The method emphasizes on producing balanced distributions of quantized values rather than preserving extremal values, by using percentiles as quantization thresholds. As a result, effective bitwidths of quantized models are increased. (See Subsection 3.2.)

2) To reduce the computation overhead introduced by computing percentiles, we approximate medians by means, which are computationally more efficient on existing hardware. The efficacy of the approximation is empirically validated. (See Subsection 3.3.)

3) Experiments confirm that our method significantly improves the prediction accuracies of CNNs and RNNs on standard datasets like ImageNet and Penn Treebank. (See Section 4.)

4) The implementation of balanced quantization will be available on-line, in TensorFlow[54] framework.

2 Quantized Neural Networks

In this section we introduce the notations and algorithms of QNNs. We also show how QNNs can exploit bitwise operations for speeding up computations and how to incorporate quantization steps into computation graphs of QNNs during training.

2.1 Notations

We will use the rounding operation intensively in this paper. For tie-breaking, we apply the "round half towards zero" rule, which rounds positive numbers with fraction $\frac{1}{2}$ down and negative numbers with fraction $\frac{1}{2}$ up. We assigns the name of "round-to-zero" for this variant of rounding:

$$\textit{round-to-zero}(x) \stackrel{\text{def}}{=\!=} \operatorname{sgn}(x)\left\lceil |x| - \frac{1}{2} \right\rceil.$$

Without loss of generality, we represent weight parameters of a neural network as a matrix $\boldsymbol{W}$. When doing k-bit uniform quantization with the step length $\frac{1}{2^k-1}$, we can define a utility function Q_k that converts floating point numbers in close interval [0, 1] to fixed point numbers as follows:

$$Q_k(\boldsymbol{W}) \stackrel{\text{def}}{=\!=} \frac{\textit{round-to-zero}((2^k-1)\boldsymbol{W})}{2^k-1}, \quad 0 \leqslant w_{i,j} \leqslant 1,\ \forall i, j. \tag{1}$$

The outputs of Q_k are the fixed point values $0,\ \frac{1}{2^k-1},\ \frac{2}{2^k-1}, \cdots,\ 1$.

In a quantized neural network, we use Q_k for the quantization of parameters, activation and gradients. When quantizing parameters, as the utility function Q_k requires the input to be in close interval [0, 1], we should first map the parameter $\boldsymbol{W}$ to the value range. The method in [51, 53] uses the following affine transform to change the value range.

Definition 1 (k-Bit Uniform Quantization).

$$\varphi(\boldsymbol{W}) \stackrel{\text{def}}{=\!=} \frac{\boldsymbol{W}}{2\max(|\boldsymbol{W}|)} + \frac{1}{2},$$
$$\textit{quant}_k(\boldsymbol{W}) \stackrel{\text{def}}{=\!=} \varphi^{-1}(Q_k(\varphi(\boldsymbol{W}))),$$

where subscript k in $\textit{quant}_k$ stands for k-bit quantization, and $|\boldsymbol{W}|$ is a matrix with values being the absolute values of corresponding entries in $\boldsymbol{W}$.

As $-\max(|\boldsymbol{W}|) \leqslant w_{i,j} \leqslant \max(|\boldsymbol{W}|)$, we have $0 \leqslant \frac{w_{i,j}}{2\max(|\boldsymbol{W}|)} + \frac{1}{2} \leqslant 1$. We can then apply Q_k to get the fixed point values $Q_k(\varphi(\boldsymbol{W}))$, which are affine transformed by φ^{-1} to restore the value range back to the closed interval $[-\max(|\boldsymbol{W}|), \max(|\boldsymbol{W}|)]$.

2.2 Simplistic View of Quantized Neural Networks

QNNs are neural networks that use quantized values for computations. Because the convolutions between inputs and convolution kernels can also be represented as matrix products, w.l.o.g., we take a multi-layer perceptron (MLP) as an example throughout the rest of this paper.

Let the outputs, activation function, weight parameters and bias parameters of the i-th layer of a neural network be $\boldsymbol{X}_i$, σ_i, $\boldsymbol{W}_i$ and $\boldsymbol{b}_i$, respectively. The i-th convolution/fully-connected layer can be represented as:

$$\boldsymbol{X}_i = \sigma_i(\boldsymbol{W}_i\boldsymbol{X}_{i-1} + \boldsymbol{b}_i).$$

The corresponding formula for the i-th convolution/fully-connected layer of a QNN is:

$$\begin{aligned} \boldsymbol{X}_i^{\mathrm{q}} &= Q_{\mathrm{A}}(\sigma_i(\boldsymbol{W}_i^{\mathrm{q}}\boldsymbol{X}_{i-1}^{\mathrm{q}} + \boldsymbol{b}_i)), \\ \boldsymbol{W}_i^{\mathrm{q}} &= Q_{\mathrm{W}}(\boldsymbol{W}_i), \end{aligned} \tag{2}$$

where $\boldsymbol{W}_i^{\mathrm{q}}$ and $\boldsymbol{X}_i^{\mathrm{q}}$ are quantized weights and activations respectively; Q_{W} and Q_{A} are quantization functions of weights and activations respectively. Note the bias parameters $\boldsymbol{b}_i$ may not need to be quantized, which we will explain in Appendix A.1.

Other types of layers like pooling layers may also take quantized values as inputs and outputs. The input to the first layer of a QNN may have higher bitwidth than the rest of the network to preserve information[51].

2.3 Exploiting Bitwise Operations in QNNs

Using quantized values for computation makes it possible to use fixed point operations instead of floating point operations. We next show how to perform dot products between quantized numbers by bitwise operations.

We first consider the dot products between k-bit fixed point numbers. In the extreme case of $k = 1$, dot products are done between bit strings, which allows for the following method of using bitwise operations:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \textit{bitcount}(\textit{and}(\boldsymbol{x}, \boldsymbol{y})), \quad \forall i, x_i, y_i \in \{0, 1\},$$

where *bitcount* counts the number of 1 in a bit string, and *and* performs bitwise AND operation.

In the multi-bit case ($k > 1$), we may also exploit the above kernel as in [42]. Assume $\boldsymbol{x}$ is a sequence of M-bit fixed point integers s.t. $\boldsymbol{x} = \sum_{m=0}^{M-1} c_m(\boldsymbol{x})2^m$ and $\boldsymbol{y}$ is a sequence of K-bit fixed point integers s.t. $\boldsymbol{y} = \sum_{k=0}^{K-1} c_k(\boldsymbol{y})2^k$ where $(c_m(\boldsymbol{x}))_{m=0}^{M-1}$ and $(c_k(\boldsymbol{y}))_{k=0}^{K-1}$ are bit vectors, the dot product of $\boldsymbol{x}$ and $\boldsymbol{y}$ can be computed by bitwise operations as:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{m=0}^{M-1}\sum_{k=0}^{K-1} 2^{m+k}\, \textit{bitcount}(\textit{and}(c_m(\boldsymbol{x}), c_k(\boldsymbol{y}))),$$
$$c_m(\boldsymbol{x})_i, c_k(\boldsymbol{y})_i \in \{0, 1\},\ \forall i,\ m,\ k.$$

In the above equation, the computation complexity is $O(MK)$, i.e., directly proportional to the product of bitwidths of $\boldsymbol{x}$ and $\boldsymbol{y}$. Hence it is beneficial to reduce the bitwidth of a QNN as long as the prediction accuracy is kept at the same level. It has been demonstrated that exploiting dot-product kernels allows for efficient software[51] and hardware implementations[55-56].

In (2), the matrix multiplication happens between the quantized values $\boldsymbol{W}_i^{\mathrm{q}}$ and $\boldsymbol{X}_{i-1}^{\mathrm{q}}$. When the activation function is monotone, the computation of $\boldsymbol{X}_i^{\mathrm{q}}$ can all be performed by operations on fixed-point numbers, even when the bias parameters $\boldsymbol{b}_i$ are floating point numbers. The method is detailed in Appendix A.1.

2.4 Training Quantized Neural Networks by Straight-Through Estimator

Having quantization steps in computation prevents the direct training of QNNs with the BP algorithm, as mathematically any quantization function will have zero derivatives. To remedy this, Courbariaux *et al.*[57] proposed to use STE to assign non-zero gradients for quantization functions. As the discrete parameters cannot be used to accumulate the high precision gradients, they kept two copies of parameters, one consisting of quantized values $\boldsymbol{W}_{\mathrm{q}}$ and the other consisting of real values $\boldsymbol{W}$. The real value version $\boldsymbol{W}$ is used for accumulation, while $\boldsymbol{W}_{\mathrm{q}}$ is used for computation in forward and backward passes. We will refer to $\boldsymbol{W}_{\mathrm{q}}$ as quantized parameters, or simply parameters, of QNNs, and reserve $\boldsymbol{W}$ for the "floating point copy" in the rest of this paper.

As STE introduces approximation noises into computations of gradients, we would like to limit it to places where necessary. It can be observed that the only function in (1) that has the zero gradients is the rounding function. Hence we construct its STE version, *round-to-zero*$_{\mathrm{ste}}$, as follows:

$$\begin{aligned} \text{forward:}\quad & \tilde{\boldsymbol{W}} \leftarrow \textit{round-to-zero}(\boldsymbol{W}), \\ \text{backward:}\quad & \frac{\partial C}{\partial \boldsymbol{W}} \leftarrow \frac{\partial C}{\partial \tilde{\boldsymbol{W}}}, \end{aligned}$$

where $\tilde{\boldsymbol{W}}$ is the rounded value and C is the objective function used in the training of the neural network.

Functions using *round-to-zero* function, like the k-bit uniform quantization function $quant_k$, can be transformed into the STE version by replacing *round-to-zero* with *round-to-zero*$_{\mathrm{ste}}$.

A QNN can then use $quant_{\mathrm{ste}}$ to include quantization in its computation graph. For completeness, we provide the inference and training algorithm of an L-layer QNN as Algorithm A1 in Appendix A.2.

3 Balanced Quantization for Neural Network Parameters

In this section, we focus on more effective quantization of parameters of QNNs to improve their prediction accuracies. We propose the Balanced Quantization method, which induces the quantized parameters to have balanced distributions. The method divides parameters by percentiles into bins containing the same number of entries before the quantization. We also propose to use approximate thresholds in Algorithm 2 in Subsection 3.3 to reduce computation overhead during training.

3.1 Effective Bitwidth and Prediction Accuracy of QNNs

Using QNNs can reduce computation resource requirements considerably. However, QNNs usually have lower prediction accuracies than their floating point counterparts, especially when bitwidths go below 4-bit[51-53].

We investigate this inefficiency of using low bitwidth parameters by inspecting the parameters of QNNs before and after the quantization. On many such models, we observe that the parameters before the quantization follow bell-shaped distributions, just as other DNNs[58-59]. Moreover, it is not rare to observe outliers. Consequently, the quantized values after uniform quantization will often follow imbalanced distributions between possible values. An illustrative example is given in Fig.1 and Fig.2, where histograms of weight parameters, before and after the quantization, of a layer in a quantized ResNet model are shown. The quantized weights are 2-bit.

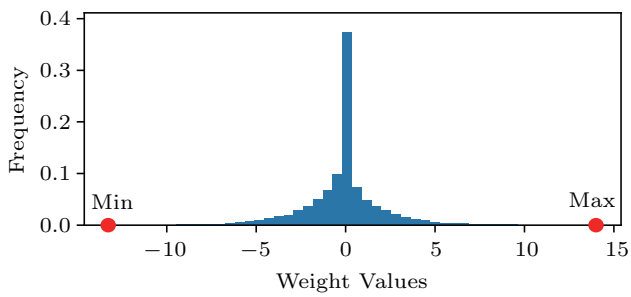


Fig.1. Floating point copy of weights in a QNN after 60 epochs of training. The weight values follow a bell-shaped distribution, and the minimum and the maximum values differ a lot from the other values.

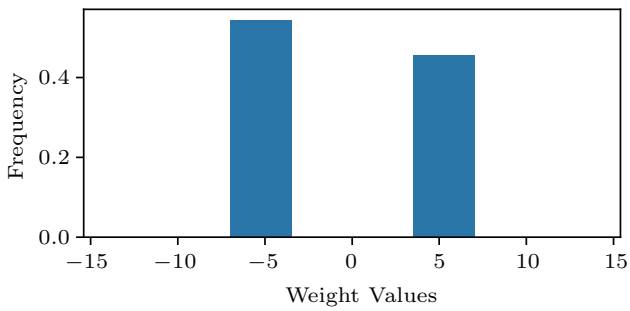


Fig.2. Results of imbalanced quantization (no equalization). After the uniform quantization of weight values to 2-bit numbers, the quantized values concentrate on the central two out of four possible quantized values.

A QNN with parameters following imbalanced distributions may be suboptimal. For example, the 2-bit weight model in Fig.2 fails to exploit available value range, and may be well approximated with a 1-bit weight model.

Hence the "real" bitwidth of a QNN may be well below its specified bitwidth. To quantitatively measure the "effective" bitwidth, we propose to use the mean of entropy of parameters of each layer in a QNN as an indicator as follows.

Definition 2.

$$\begin{aligned} &\textit{effective-bitwidth}(X) \stackrel{\text{def}}{=\!=} \textit{entropy}(\Pr(X)) \\ &= \textit{bitwidth} \times \frac{\textit{entropy}(\Pr(X))}{\textit{entropy}(\textit{UniformDistribution})}, \end{aligned}$$

where entropy is defined with base-2 logarithm, and $\Pr(X)$ *refers to the distribution of* X.

The definition is in agreement with the following intuitions.

1) If the quantized values are concentrated in a few bins like in Fig.2, indicating the poor utilization of the available bitwdith, the effective bitwidth will be low, just as expected.

2) When the order of the bars standing for quantized values in histogram is permuted, which does not increase bitwidth utilization, the effective bitwidth will not change.

3) If X is drawn from a discrete uniform distribution with 2^B possible values, then *effective-bitwidth*$(X) = B$ as desired.

Based on this definition of effective bitwidth, we make the following conjecture that will be empirically validated in Subsection 4.1.2.

Conjecture 1. *Assume other factors affecting prediction accuracies, like learning rate schedules and model architectures, are kept the same. The prediction accuracy of a converged QNN model is positively correlated with its effective bitwidth.*

Motivated by the conjecture, we propose a novel quantization algorithm that can enforce the balanced distribution of quantized parameters, which maximizes the entropy of the converged model and consequently its effective bitwidth.

3.2 Balanced Quantization Algorithm

3.2.1 Outline

In this subsection we propose an algorithm to induce parameters of QNNs to have more balanced distributions, and consequently larger effective bitwidths.

The first step is histogram equalization, which can be implemented as a piecewise linear transform. The second step performs quantization, and then matches the value range with that of the input by an affine transformation.

Fig.3 gives a schematic diagram of the Balanced Quantization algorithm. The method starts by partitioning numbers into bins containing the same number of entries. Each partition is then mapped to an evenly-divided interval in the closed interval [0, 1]. Finally,

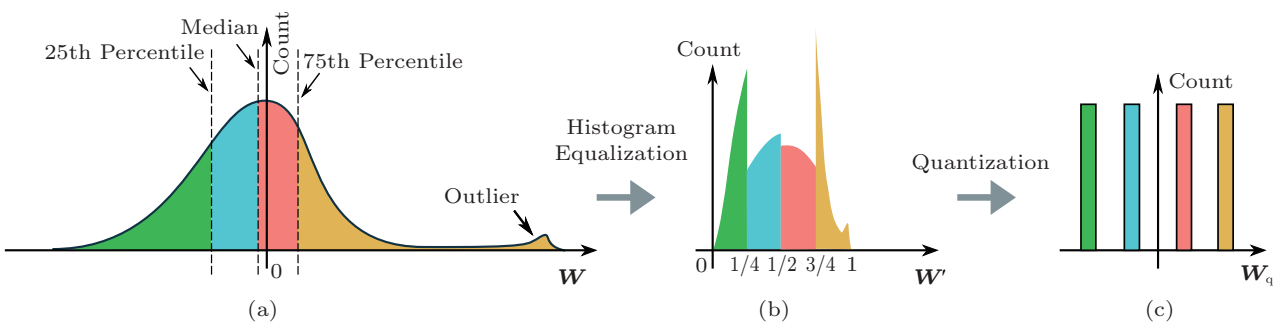


Fig.3. Schematic description of the Balanced Quantization algorithm in presence of outliers, with the case of $k = 2$ as an example. The histogram of the weight values is first equalized by piecewise linear transform and then mapped to a symmetric distribution. (a) Histogram of floating-point weight values. (b) Histogram-equalized weight values. (c) Quantized weight values.

the quantization step maps intervals into discrete values and transforms the value range to be approximately the same with the input. There will be exactly the same number of quantized values assigned to possible choices when percentiles are used as thresholds.

Algorithm 1 gives a more rigorous description of the whole process.

Algorithm 1. k-Bit Balanced Quantization Algorithm of Matrix $\boldsymbol{W}$

Require: $\boldsymbol{W}$ is a real matrix
Ensure : $\boldsymbol{W}_{\mathrm{q}}$ is quantized weights

1 $scale \leftarrow \max(|\boldsymbol{W}|)$;

{`Histogram Equalization`}

{`The equalized values` $\boldsymbol{W}_{\mathrm{e}}$ `are in closed interval` $[0,\ 1]$}

2 $\boldsymbol{W}_{\mathrm{e}} \leftarrow equalize_k(\boldsymbol{W})$;

{`Quantization and restoring value range`}

{$\boldsymbol{W}_{\mathrm{f}}$ `are fixed point numbers among` 2^k `discrete values` $-\frac{1}{2}, -\frac{1}{2}+\frac{1}{2^k-1}, -\frac{1}{2}+\frac{2}{2^k-1}, \cdots, \frac{1}{2}$}

3 $\boldsymbol{W}_{\mathrm{f}} \leftarrow \frac{1}{2^k-1} \textit{round-to-zero}(2^k\boldsymbol{W}_{\mathrm{e}} - \frac{1}{2}) - \frac{1}{2}$;

{`Values of` $\boldsymbol{W}_{\mathrm{q}}$ `are scaled fixed point numbers in closed interval` $[-\max(|\boldsymbol{W}|),\ \max(|\boldsymbol{W}|)]$}

4 $\boldsymbol{W}_{\mathrm{q}} \leftarrow 2 \times scale \cdot \boldsymbol{W}_{\mathrm{f}}$;

3.2.2 Histogram Equalization by Piecewise Linear Transform

In this subsection, we detail the histogram equalization step, which we adapt from image processing literature[60].

Assume we are quantizing to k-bit values and $N = 2^k$. The input value range is divided to N intervals, including $N-1$ numbers of half open intervals $[t_i, t_{i+1})$ and a closed interval $[t_{N-1}, t_N]$. To simplify notation, we denote the i-th interval as I_i. Thresholds $\{t_i\}_{i=0}^N$ are determined by the algorithm of histogram equalization. When exact equalization is desired, we let thresholds t_i be the $\frac{100i}{N}$-th percentiles of the original distribution. The formula of equalized values x_e is as follows.

Definition 3 (Histogram Equalization).

$$x_{\mathrm{e}} = equalize_k(x) \stackrel{\text{def}}{=\!=} a_i x + b_i,$$
$$if\ x \in I_i, \quad 0 \leqslant i \leqslant N-1,\ i \in \mathbb{Z}.$$

As $equalize_k$ maps I_i to evenly spaced segments J_i of target interval $[0, 1]$, parameters of the affine transformations, a_i and b_i, can be determined from the following constraints:

$$a_i t_i + b_i = \frac{i}{N},$$

$$a_i t_{i+1} + b_i = \frac{i+1}{N}, \quad 0 \leqslant i \leqslant N-1,$$

where $\frac{i}{N}$ and $\frac{i+1}{N}$ are the two endpoints of J_i.

Let C be the objective function of the training, and the back-propagation formula for $equalize_k$ is straightforward:

$$\frac{1}{a_i}\frac{\partial C}{\partial x} = \frac{\partial C}{\partial x_{\mathrm{e}}},$$
$$\text{if } x_{\mathrm{e}} \in J_i, \quad 0 \leqslant i \leqslant N-1,\ i \in \mathbb{Z}.$$

3.2.3 Rounding and Restoring Value Range

After the histogram equalization step, the values $\boldsymbol{W}_{\mathrm{e}}$ are still floating point values, and need to be converted to discrete values. The conversion can be done by the construction of fixed point version $\boldsymbol{W}_{\mathrm{f}} = \frac{1}{2^k-1}\textit{round-to-zero}(2^k\boldsymbol{W}_{\mathrm{e}} - \frac{1}{2}) - \frac{1}{2}$. Note the mapping between $\boldsymbol{W}_{\mathrm{e}}$ and $\boldsymbol{W}_{\mathrm{f}}$ is different from Q_k. For example, it maps the interval of $[0,\ \frac{1}{2^k}]$ to 0, while Q_k maps $[0,\ \frac{1}{2(2^k-1)}]$ to 0.

Finally, $\boldsymbol{W}_{\mathrm{f}}$, which has value range $[-\frac{1}{2}, \frac{1}{2}]$, can be scaled by $2\max(|\boldsymbol{W}|)$ to match the original value range.

3.3 Approximation of Medians and Efficient Implementation

The histogram equalization defined by the piecewise linear transform in Definition 3 has well-defined gradients and can be readily integrated into the training process of QNNs. However, a naive implementation using percentiles as thresholds would require sorting of weight values during each forward operation in BP, which may slow down the training process of QNNs as sorting is less efficient on modern hardware than matrix multiplications. In this subsection, we discuss an approximate equalization that allows efficient implementation. We first propose a recursive implementation of histogram equalization that only requires computing medians. Noting that medians can be well approximated by means, we construct Algorithm 2 that can perform approximate histogram equalization without doing sorting.

3.3.1 Recursive Partitioning

We first note that the 2^k evenly spaced percentiles required in histogram equalization can be computed from the recursive application of partitioning of numbers by medians. For example, when doing histogram equalization for the 2-bit quantization, we need to compute the 25-th, the 50-th and the 75-th percentiles as thresholds. However, the 50-th percentile is exactly the

median, while the 25-th percentile (25% of values are below this number) is the median of those values that are below the median of the original distribution. Hence we can replace the computation of percentiles with recursive applications of partitioning by medians.

Algorithm 2. Histogram Equalization of Matrix $\boldsymbol{W}$ by Recursive Partitioning

1 **Function** `HistogramEqualize(`$\boldsymbol{W}$, $\boldsymbol{M}$, *level*`)` **Data**:
  $\boldsymbol{W}$ is a real-valued matrix;
  $\boldsymbol{M}$ is a mask matrix with values in $\{0, 1\}$ and has the same shape as $\boldsymbol{W}$. It is used to note the "working set" of $\boldsymbol{W}$.
  level is an auxiliary variable recording recursion level.
  Result: a matrix of the same shape as $\boldsymbol{W}$ with value range [0, 1]

  {$S_{\boldsymbol{W}}$ `is the subset of the elements of` $\boldsymbol{W}$ `with positive masks`}
2 $S_{\boldsymbol{W}} \leftarrow \{w_{i,j} | \forall\, w_{i,j} \in \boldsymbol{W}; m_{i,j} > 0\}$
3 **if** $level = 0$ **then**
  {`Affine transform` $\boldsymbol{W}$ `to the value range of` $[0, 1]$}
  {$*$ `is element-wise (Hadamard) multiplication`}
4 **return** $\dfrac{\boldsymbol{W} - \min(S_{\boldsymbol{W}})}{\max(S_{\boldsymbol{W}}) - \min(S_{\boldsymbol{W}})} \circ \boldsymbol{M}$
5 **end**

  {`Construct two masks` $\boldsymbol{M}^l$ `and` $\boldsymbol{M}^g$ `using` $\mathrm{mean}(S_{\boldsymbol{W}})$ `as threshold`}
  {$\mathrm{mean}(S_{\boldsymbol{W}})$ `is used to replace` $\mathrm{median}(S_{\boldsymbol{W}})$ `so as to accelerate computation (see Subsection 3.3)`}
6 $T \leftarrow \mathrm{mean}(S_{\boldsymbol{W}}) = \frac{\sum(\boldsymbol{W} \circ \boldsymbol{M})}{\sum \boldsymbol{M}}$
7 $\boldsymbol{M}^l \leftarrow \boldsymbol{0}$, $\boldsymbol{M}^g \leftarrow \boldsymbol{0}$
8 **for** $w_{i,j} \in \boldsymbol{W}$; $m_{i,j} > 0$ **do**
9 **if** $w_{i,j} < T$ **then**
10 $m^l_{i,j} \leftarrow 1$
11 **else**
12 $m^g_{i,j} \leftarrow 1$
13 **end**
14 **end**
15 $\boldsymbol{W}^l \leftarrow$ `HistogramEqualize(`$\boldsymbol{W}$, $\boldsymbol{M}^l$, $level - 1$`)`
16 $\boldsymbol{W}^g \leftarrow$ `HistogramEqualize(`$\boldsymbol{W}$, $\boldsymbol{M}^g$, $level - 1$`)`
  {`Value ranges of both` $\boldsymbol{W}^l$ `and` $\boldsymbol{W}^g$ `are` $[0, 1]$}
  {$\frac{1}{2}$ `is added to` $\frac{1}{2}\boldsymbol{W}^g$ `to shift the value range to` $[\frac{1}{2}, 1]$}
17 **return** $\frac{1}{2}\boldsymbol{W}^l + (\frac{1}{2}\boldsymbol{W}^g + \frac{1}{2}) \circ \boldsymbol{M}^g$

Moreover, we note that when a distribution has bounded variance σ, the mean μ approximates the median m as there is an inequality bounding the difference[61]:

$$|\mu - m| \leqslant \sigma.$$

Hence we may use means instead of medians in the recursive partitioning. The results of partitioning by different methods are shown in Fig.4 and Fig.5. It can be observed that partitioning by the median achieves perfect balance, and partitioning by the mean achieves approximate balance.

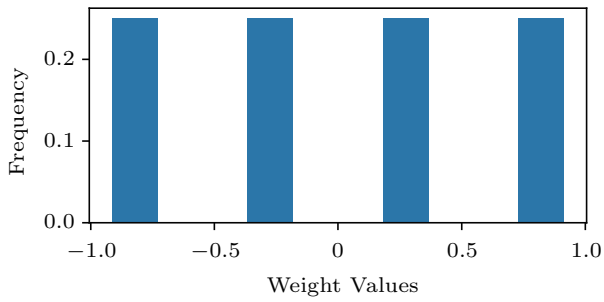


Fig.4. Balanced quantization with medians (before matching value range).

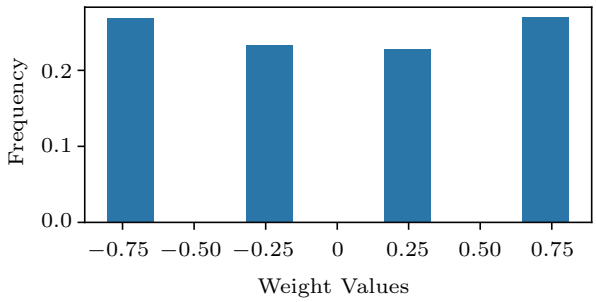


Fig.5. Balanced quantization with means (before matching value range).

3.3.2 Implementation

Based on the fact that the median approximates the mean, histogram equalization can be implemented as in Algorithm 2. An auxiliary mask matrix $\boldsymbol{M}$, whose values are either 0 or 1, is introduced to help manipulate the branching and selection operations. Note the mask $\boldsymbol{M}$, which is an argument of `HistogramEqualize` at the top of call chain, is initialized to be $\boldsymbol{1}$, a matrix with all values being 1.

When Algorithm 2 is used as the histogram equalization step in Algorithm 1, we can prove the following proposition (see Appendix A.3 for the proof).

Proposition 1. *If during the application of Algorithm* 2 *the following holds after line* 14:

$$\frac{1}{\gamma} \leqslant \frac{\sum \boldsymbol{M}^l}{\sum \boldsymbol{M}^g} \leqslant \gamma,$$

then the most frequent entry of the quantized values will appear at most γ^{2K} *as often as that of the least frequent entry, when quantizing to* K*-bit numbers with Algorithm* 1.

4 Experiments

In this section we empirically validate the effectiveness of the balanced quantization through experiments on quantized convolutional neural networks and recurrent neural networks.

In our implementations of QNNs, we convert parameters and input activations of all layers in the network to low bitwidth numbers, which is in line with the practice of Hubara *et al.*[51] The CNN models used in this section are all equipped with batch normalization[62] to speed up convergence. Experiments are done on Linux machines with Intel® Xeon® CPUs and NVidia Titan X graphic processing units.

4.1 Experiments on Convolutional Neural Networks

4.1.1 Datasets

For the evaluation on CNNs, we conduct experiments on two datasets used for the image classification task.

The SVHN dataset[63] is a real-world digit recognition dataset consisting of photos of house numbers in Google Street View images. We consider the "cropped" format of the dataset: 32-by-32 colored images centered around a single character. We also include the "extra" part of labeled data in training.

The ImageNet dataset contains 1.2M images for training and 50k images for validation. Each image in the dataset is assigned a label in one of the 1 000 categories. While testing, images are first resized such that the shortest edge is 256 pixels, and then the center 224-by-224 crops are fed into models. Following the conventions, we report results in two measures: single-crop top-1 error rate and top-5 error rate over ILSVRC12 validation sets[64]. For brevity, we will denote the top-1 and top-5 error rates as "top-1" and "top-5", respectively.

4.1.2 Effective Bitwdiths and Prediction Accuracies of Converged Models

In Fig.6 and Fig.7, we plot the prediction accuracies of several converged QNNs against their effective bitwidths as defined in Definition 2. QNNs are trained on the SVHN dataset and have the same 7-layer CNN model architecture; hyper-parameters like learning rate schedule, numbers of epochs are kept the same, such that the differences between these models are only the specified bitwidths of parameters and the quantization methods. In this way, we can evaluate the impact of effective bitwidths on the prediction accuracies of converged models.

It can be observed from Fig.6 that in general, prediction accuracy grows with the increase of effective bitwidths. However, the growth of the accuracy gradually slows down to the right half of the diagram, when the prediction accuracy of a quantized model approaches the upper bound set by floating point models.

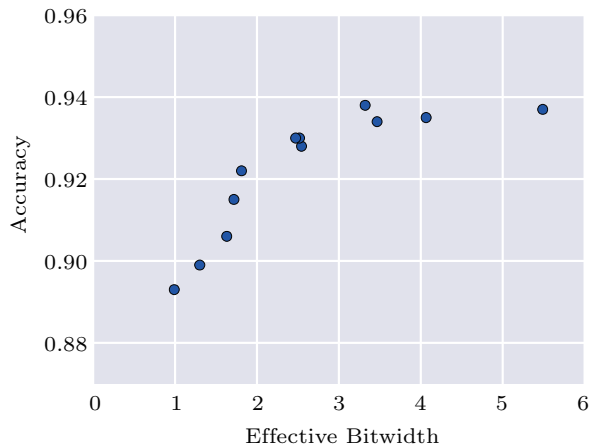


Fig.6. Relationship between effective bitwidths and prediction accuracies of several converged QNNs on the SVHN dataset. The models are produced by different specified bitwidths (ranging from 1-bit to 8-bit) and quantization methods (balanced or not), but they all have the same architecture and training settings.

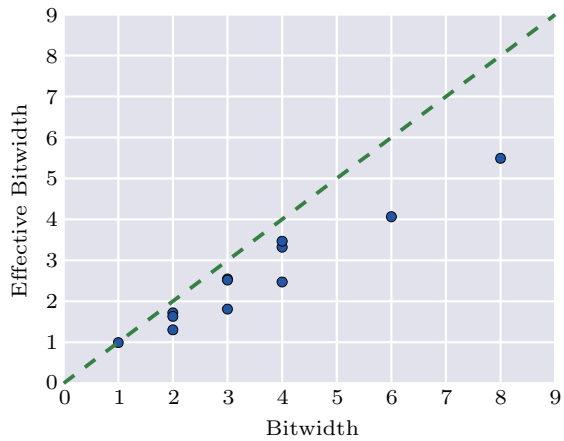


Fig.7. Relationship between effective bitwidth and specified bitwidth. In general, the effective bitwidth grows with the bitwidth. But the effective bitwidths of most of the models are significantly less than their specified bitwidths.

4.1.3 Evaluation of Approximation of Medians

In this subsection we validate the effectiveness of approximation of the median by the mean, as pro-

posed in Subsection 3.3. As computing the median requires doing sorting of weight parameters of a layer, experiments on DNNs with many parameters will be very slow. Hence we perform experiments on the GoogLeNet, which contains fewer than 7M parameters. From Table 1, it can be seen that replacing medians by means does not degrade prediction accuracies. In fact, the method using means as thresholds is even slightly better than the method using medians, both in terms of top-1 and top-5 error rates.

Table 1. Evaluation of Using Means Instead of Medians When Performing Balanced Quantization, on GoogLeNet with 4-Bit Weights and 4-Bit Activations

Threshold	Top-1 (%)	Top-5 (%)	Effective Bitwidth
Mean	32.3	12.7	3.99
Median	33.8	13.3	4.00

As replacing medians with means is empirically found to be viable, we will use means as thresholds in experiments in the rest of Section 4.

4.1.4 *Balanced Quantization of AlexNet, ResNet-18 and GoogLeNet*

The experimental results on AlexNet and ResNet-18 are summarized in Table 2. The results marked with "different settings" come from models trained with a different learning rate schedule and clipping of weights. It can be seen that results of the balanced quantization method consistently outperform those of the uniform quantization methods (hereafter denoted as imbalanced quantization) defined in Definition 1. In particular, the top-5 error rate of the Balanced Quantized 2-bit AlexNet is within 2% of that of the floating point version, making the quantized network a good candidate to replace the floating point version in practice. As the model size can be reduced to $\frac{1}{16}$ of the original and computations can be performed by 2-bit numbers, the savings in resource requirements will be significant.

However, the improvements of accuracies due to balanced quantization may not be large, as accuracies of models quantized without balance are already close to the upper bounds set by models with floating point weights and 2-bit features.

Table 3 compares QNNs quantized with our method with the state-of-the-art QNNs. It can be seen that our method consistently outperforms the others. In particular, our method reduces the top-5 accuracy degradation, which is the difference in the accuracy between a QNN and a floating point version, from 6.5% to 2.6%.

4.1.5 *Break-Down of Accuracy Degradation with Balanced Quantization*

Overall, the change in accuracy due to balanced quantization will be made up of two parts:

$$\Delta Accuracy_{\text{total}} = \Delta Accuracy_{\text{eq}} + \Delta Accuracy_{\text{quant}},$$

where $\Delta Accuracy_{\text{eq}}$ stands for the change in accuracy due to equalization and $\Delta Accuracy_{\text{quant}}$ is that of quantization.

The histogram equalization effectively imposes an additional constraint on the neural network parameters. As the constraint limits the optimization space of parameters, it will likely introduce additional errors into predictions of neural networks.

Nevertheless, through the experiments in Table 2, we have observed that the reduction in $\Delta Accuracy_{\text{quant}}$ outweighs the inclusion of additional term $\Delta Accuracy_{\text{eq}}$. We leave it as future work to investigate the cause and further the reduction of $\Delta Accuracy_{\text{eq}}$.

Table 2. Comparison of Performances of Quantized AlexNet and ResNet-18

Method	AlexNet			ResNet-18		
	Top-1 (%)	Top-5 (%)	Effective Bitwidth	Top-1 (%)	Top-5 (%)	Effective Bitwidth
FP	42.9	20.6	-	31.8	12.5	-
Equalized FP weights	42.7	20.9	-	36.2	15.3	-
FP weight + 2-bit feature	43.5	21.0	-	38.9	17.3	-
Imbalanced 2-bit (different settings)	46.4	24.7	1.89	46.6	22.1	0.99
Imbalanced 2-bit	45.3	22.3	1.94	42.3	19.2	1.96
Balanced 2-bit	**44.3**	**22.0**	1.99	**40.6**	**18.0**	1.99

Note: FP stands for floating point. Results in rows prefixed with "imbalanced" are produced from direct applications of uniform quantization. Results in the row marked with "equalized FP weights" are produced by only performing the equalization of weights on FP models. As the floating point values do not have well-defined effective bitwidths, we omit these entries by using the "-" symbol.

Table 3. Comparison of Classification Error Rates with the State of the Arts on Quantized GoogLeNet Model with 4-Bit Weights and 4-Bit Activations

Method	Top-1 (%)	Top-5 (%)
Our float32	28.5	10.1
QNN 4-bit[51]	33.5	16.6
Ristretto 8-bit[65]	33.4	-
Our 4-bit	**32.3**	**12.7**

4.2 Experiments on Recurrent Neural Networks

In this subsection we evaluate the effect of balanced quantization on a few recurrent neural networks. We take language modeling task as an example, and use the Penn Treebank (PTB) dataset[66], which contains 10k unique words.

For fair comparison, in the following experiments, all of our models use one hidden layer with 300 hidden units, which is the same with [51]. A word embedding layer is used at the input side of the network whose weights are trained from scratch. The performance is measured in perplexity per word (PPW) metric.

During experiments we find the magnitudes of weights often grow rapidly with training when using small bitwidth, and may result in divergence. This can be alleviated by adding tanh to constrain the value ranges[53] and adding weight decays for regularization. However, we find using tanh to clip parameters will degrade prediction accuracy of floating point neural network. Further investigation of this drop of accuracy is out of the scope of this paper, and will be left as future work.

Experimental results are reported in Table 4. Our result is in agreement with [51] in finding that using 4-bit weights and activations can achieve comparable performance as floating point counterparts. However, we report a higher accuracy than [51] when using less bits, such as 2-bit weights and activations. In particular, our 2-bit weights and 3-bit activations LSTM (long short-term memory) achieve 155 PPW for imbalanced quantization and 123 PPW for balanced quantization, both of which outperform the counterparts in [51] by large margins, despite that our floating point models are worse than those of [51].

5 Conclusions

In this paper, we introduced the method of balanced quantization, which enforces the quantized values to have balanced distributions through the use of histogram equalization. Our method breaks away from traditional quantization methods in that it emphasizes on shaping distributions of quantized values. When incorporated into the training process of quantized neural networks, our method can improve the prediction accuracies of converged models. We also introduced effective bitwidth, which measures the utilization of bitwidth in QNNs, that can help identify models that can benefit more from the balanced quantization method.

To reduce the computation overhead introduced by the need to compute percentiles when performing balanced quantization, we also proposed to use the re-

Table 4. Performance of Quantized RNNs on PTB Datasets

Model	W-Bit	A-Bit	PPW		Effective Bitwidth	
			Balanced	Imbalanced	Balanced	Imbalanced
GRU	2	2	142	165	1.98	1.56
GRU	4	4	116	120	3.86	3.26
GRU (tanh($\boldsymbol{W}$))	FP	FP	-	118	-	-
GRU	FP	FP	-	100	-	-
LSTM	2	2	**126**	164	1.96	1.00
LSTM	2	3	**123**	155	1.95	1.00
LSTM[51]	2	3		220		
LSTM	4	4	114	127	3.89	1.80
LSTM[51]	4	4		**100**		
LSTM (tanh($\boldsymbol{W}$))	FP	FP	-	122	-	-
LSTM	FP	FP	-	106	-	-
LSTM[51]	FP	FP		**97**		

Note: FP stands for 32-bit floating point. Results marked with tanh($\boldsymbol{W}$) are of models that have their weights clipped by tanh before passing to quantization. The best results for each bitwidth setting are marked in bold.

cursive application of means as approximations of percentiles (see Subsection 3.3). We also applied the Balanced Quantization method to several popular neural network architectures like AlexNet, GoogLeNet and ResNet, and found that our method outperforms the state-of-the-arts of QNN, in terms of prediction accuracy (see Subsection 4.1.4). Experiments on LSTM and GRU are also encouraging (see Subsection 4.2).

As future work, it would be interesting to use the histogram transformation technique to induce distributions that have other benefits, like a high ratio of zeros in quantized values. It would also be interesting to investigate whether inducing activations of neural networks to have balanced distributions could improve the prediction accuracies of QNNs.

References

[1] Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks. In *Proc. the 25th Int. Conf. Neural Information Processing Systems*, December 2012, pp.1097-1105.

[2] Zeiler M D, Fergus R. Visualizing and understanding convolutional networks. In *Proc. European Conference on Computer Vision*, September 2014, pp.818-833.

[3] Girshick R, Donahue J, Darrell T, Malik J. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, June 2014, pp.580-587.

[4] Long J, Shelhamer E, Darrell T. Fully convolutional networks for semantic segmentation. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, June 2015, pp.3431-3440.

[5] Hinton G, Deng L, Yu D, Dahl G E, Mohamed A R, Jaitly N, Senior A, Vanhoucke V, Nguyen P, Sainath T N, Kingsbury B. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 2012, 29(6): 82-97.

[6] Graves A, Mohamed A R, Hinton G E. Speech recognition with deep recurrent neural networks. In *Proc. IEEE Int. Conf. Acoustics Speech and Signal Processing* (*ICASSP*), May 2013, pp.6645-6649.

[7] Mikolov T, Sutskever I, Chen K, Corrado G S, Dean J. Distributed representations of words and phrases and their compositionality. In *Proc. Advances in Neural Information Processing Systems*, December 2013, pp.3111-3119.

[8] Sutskever I, Vinyals O, Le Q V. Sequence to sequence learning with neural networks. In *Proc. Advances in Neural Information Processing Systems*, December 2014, pp.3104-3112.

[9] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. *arXiv: 1409.0473*, 2014. http://arxiv.org/abs/1409.0473, May 2017.

[10] Mnih V, Kavukcuoglu K, Silver D, Rusu A A, Veness J, Bellemare M G, Graves A, Riedmiller M, Fidjeland A K, Ostrovski G, Petersen S, Beattie C, Sadik A, Antonoglou I, King H, Wierstra D K D, Legg S, Hassabis D. Human-level control through deep reinforcement learning. *Nature*, 2015, 518(7540): 529-533.

[11] Silver D, Huang A, Maddison C J, Guez A, Sifre L, van den Driessche G, Schrittwieser J, Antonoglou I, Panneershelvam V, Lanctot M, Dieleman S, Grewe D, Nham J, Kalchbrenner N, Sutskever I, Lillicrap T, Leach M, Kavukcuoglu K, Graepel T, Hassabis D. Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016, 529(7587): 484-489.

[12] He K M, Zhang X Y, Ren S Q Sun J. Identity mappings in deep residual networks. In *Proc. the 14th European Conf. Computer Vision* (*ECCV*), October 2016, pp.630-645.

[13] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. *arXiv: 1409.1556*, 2014. http://arxiv.org/abs/1409.1556, May 2017.

[14] Szegedy C, Liu W, Jia Y Q, Sermanet P, Reed S E, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A. Going deeper with convolutions. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, June 2015.

[15] He K M, Zhang X Y, Ren S Q, Sun J. Deep residual learning for image recognition. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, June 2016, pp.770-778.

[16] Galal S, Horowitz M. Energy-efficient floating-point unit design. *IEEE Trans. Computers*, 2011, 60(7): 913-922.

[17] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Computation*, 1997, 9(8): 1735-1780.

[18] Chung J, Gulcehre C, Cho K, Bengio Y. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv: 1412.3555*, 2014. http://arxiv.org/abs/1412.3555, May 2017.

[19] Pham P H, Jelaca D, Farabet C, Martini B, LeCun Y, Culurciello E. NeuFlow: Dataflow vision processing system-on-a-chip. In *Proc. the 55th IEEE Int. Midwest Symp. Circuits and Systems* (*MWSCAS*), August 2012, pp.1044-1047.

[20] Chen T S, Du Z D, Sun N H, Wang J, Wu C Y, Chen Y J, Temam O. DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proc. the 9th Int. Conf. Architectural Support for Programming Languages and Operating Systems* (*ASPLOS*), March 2014, pp.269-284.

[21] Luo T, Liu S L, Li L, Wang Y Q, Zhang S J, Chen T S, Xu Z W, Temam O, Chen Y J. DaDianNao: A neural network supercomputer. *IEEE Trans. Computers*, 2017, 66(1): 73-88.

[22] Denton E L, Zaremba W, Bruna J, LeCun Y, Fergus R. Exploiting linear structure within convolutional networks for efficient evaluation. In *Proc. the 27th Int. Conf. Neural Information Processing Systems*, December 2014, pp.1269-1277.

[23] Jaderberg M, Vedaldi A, Zisserman A. Speeding up convolutional neural networks with low rank expansions. In *Proc. British Machine Vision Conference* (*BMVC*), September 2014.

[24] Tai C, Xiao T, Zhang Y, Wang X G, E W N. Convolutional neural networks with low-rank regularization. *arXiv: 1511.06067*, 2015. http://arxiv.org/abs/1511.06067, May 2017.

[25] Zhou S C, Wu J N, Wu Y X, Zhou X Y. Exploiting local structures with the Kronecker layer in convolutional networks. *arXiv: 1512.09194*, 2015. https://arxiv.org/abs/1512.09194, May 2017.

[26] Novikov A, Podoprikhin D, Osokin A, Vetrov D. Tensorizing neural networks. In *Proc. Advances in Neural Information Processing Systems*, December 2015, pp.442-450.

[27] Zhang X Y, Zou J H, He K M, Sun J. Accelerating very deep convolutional networks for classification and detection. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 2016, 38(10): 1943-1955.

[28] Anwar S, Hwang K, Sung W. Structured pruning of deep convolutional neural networks. *arXiv: 1512.08571*, 2015. http://arxiv.org/abs/1512.08571, May 2017.

[29] Han S, Pool J, Tran J, Dally W J. Learning both weights and connections for efficient neural network. In *Proc. Advances in Neural Information Processing Systems*, December 2015, pp.1135-1143.

[30] Han S, Mao H, Dally W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv: 1510.00149*, 2015. https://arxiv.org/abs/1510.00149, May 2017.

[31] Liu B Y, Wang M, Foroosh H, Tappen M, Penksy M. Sparse convolutional neural networks. In *Proc. IEEE Conf. Computer Vision and Pattern Recognition* (*CVPR*), June 2015, pp.806-814.

[32] Cheng Y, Yu F X, Feris R S, Kumar S, Choudhary A, Chang S F. An exploration of parameter redundancy in deep networks with circulant projections. In *Proc. IEEE Int. Conf. Computer Vision*, December 2015, pp.2857-2865.

[33] Chen W L, Wilson J T, Tyree S, Weinberger K Q, Chen Y X. Compressing neural networks with the hashing trick. In *Proc. the 32nd Int. Conf. Int. Machine Learning*, July 2015, pp.2285-2294.

[34] Chen W L, Wilson J, Tyree S, Weinberger K Q, Chen Y X. Compressing convolutional neural networks in the frequency domain. In *Proc. the 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, August 2016, pp.1475-1484.

[35] Anguita D, Carlino L, Ghio A, Ridella S. A FPGA core generator for embedded classification systems. *Journal of Circuits Systems and Computers*, 2011, 20(2): 263-282.

[36] Vanhoucke V, Senior A, Mao M Z. Improving the speed of neural networks on CPUs. In *Proc. Deep Learning and Unsupervised Feature Learning Workshop*, December 2011.

[37] Alvarez R, Prabhavalkar R, Bakhtin A. On the efficient representation and execution of deep acoustic models. In *Proc. the 17th Annual Conf. the Int. Speech Communication Association*, September 2016, pp.2746-2750.

[38] Zen H, Agiomyrgiannakis Y, Egberts N, Henderson F, Szczepaniak P. Fast, compact, and high quality LSTM-RNN based statistical parametric speech synthesizers for mobile devices. In *Proc. the 17th Annual Conf. the Int. Speech Communication Association*, September 2016, pp.2273-2277.

[39] Gong Y C, Liu L, Yang M, Bourdev L. Compressing deep convolutional networks using vector quantization. *arXiv: 1412.6115*, 2014. https://arxiv.org/abs/1412.6115, May 2017.

[40] Merolla P, Appuswamy R, Arthur J, Esser S K, Modha D. Deep neural networks are robust to weight binarization and other non-linear distortions. *arXiv: 1606.01981*, 2016. https://arxiv.org/abs/1606.01981, May 2017.

[41] Gupta S, Agrawal A, Gopalakrishnan K, Narayanan P. Deep learning with limited numerical precision. *arXiv: 1502.02551*, 2015. http://arxiv.org/abs/1502.02551, May 2017.

[42] Courbariaux M, Bengio Y. BinaryNet: Training deep neural networks with weights and activations constrained to +1 or −1. *arXiv: 1602.02830v1*, 2016. http://arxiv.org/abs/1602.02830v1, May 2017.

[43] Wu J X, Leng C, Wang Y H, Hu Q H, Cheng J. Quantized convolutional neural networks for mobile devices. *arXiv: 1512.06473*, 2016. https://www.arxiv.org/abs/1512.06473, May 2017.

[44] Kim M, Smaragdis P. Bitwise neural networks. *arXiv: 1601.06071*, 2016. https://arxiv.org/abs/1601.06071, May 2017.

[45] Hubara I, Courbariaux M, Soudry D, El-Yaniv R, Bengio Y. Binarized neural networks. In *Proc. the 30th Conf. Neural Information Processing Systems*, December 2016, pp.4107-4115.

[46] Rastegari M, Ordonez V, Redmon J, Farhadi A. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Proc. the 14th European Conf. Computer Vision*, October 2016, pp.525-542.

[47] Hinton G, Srivastava N, Swersky K. Coursera: Neural networks for machine learning. 2012. https://www.class-central.com/mooc/398/coursera-neural-networks-for-machine-learning, May 2017.

[48] Bengio Y, Léonard N, Courville A C. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv: 1308.3432*, 2013. http://adsabs.harvard.edu/abs/2013arXiv1308.3432B, May 2017.

[49] Hwang K, Sung W. Fixed-point feedforward deep neural network design using weights +1, 0, and −1. In *Proc. IEEE Workshop on Signal Processing Systems*, October 2014.

[50] Shin S, Hwang K, Sung W. Fixed-point performance analysis of recurrent neural networks. In *Proc. IEEE Int. Conf. Acoustics Speech and Signal Processing* (*ICASSP*), March 2016, pp.976-980.

[51] Hubara I, Courbariaux M, Soudry D, El-Yaniv R, Bengio Y. Quantized neural networks: Training neural networks with low precision weights and activations. *arXiv: 1609.07061*, 2016. http://arxiv.org/abs/1609.07061, May 2017.

[52] Miyashita D, Lee E H, Murmann B. Convolutional neural networks using logarithmic data representation. *arXiv: 1603.01025*, 2016. https://arxiv.org/abs/1603.01025, May 2017.

[53] Zhou S C, Wu Y X, Ni Z K, Zhou X Y, Wen H, Zou Y H. DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv: 1606.06160*, 2016. https://www.arxiv.org/abs/1606.06160, May 2017.

[54] Abadi M, Agarwal A, Barham P, Brevdo E, Chen Z F, Citro C, Corrado G S, Davis A, Dean J, Devin M, Ghemawat S, Goodfellow I, Harp A, Irving G, Isard M, Jia Y Q, Jozefowicz R, Kaiser L, Kudlur M, Levenberg J, Mane D, Monga R, Moore S, Murray D, Olah C, Schuster M, Shlens J, Steiner B, Sutskever I, Talwar K, Tucker P, Vanhoucke V, Vasudevan V, Viegas F, Vinyals O, Warden P, Wattenberg M, Wicke M, Yu Y, Zheng X Q. TensorFlow: Large-scale machine learning on heterogeneous distributed systems. *arXiv: 1603.04467*, 2015. https://arxiv.org/abs/1603.04467, May 2017.

[55] Andri R, Cavigelli L, Rossi D, Benini L. YodaNN: An ultra-low power convolutional neural network accelerator based on binary weights. In *Proc. IEEE Computer Society Annual Symposium on VLSI*, July 2016, pp.236-241.

[56] Lee M, Hwang K, Park J, Choi S, Shin S, Sung W. FPGA-based low-power speech recognition with recurrent neural networks. In *Proc. IEEE Int. Workshop on Signal Processing Systems*, October 2016, pp.230-235.

[57] Courbariaux M, Bengio Y, David J P. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Proc. the 28th Int. Conf. Neural Information Processing Systems*, December 2015, pp.3123-3131.

[58] Saxe A M, Koh P W, Chen Z H, Bhand M, Suresh B, Ng A Y. On random weights and unsupervised feature learning. In *Proc. the 28th Int. Conf. Machine Learning*, June 2011, pp.1089-1096.

[59] Giryes R, Sapiro G, Bronstein A M. Deep neural networks with random gaussian weights: A universal classification strategy? *IEEE Trans. Signal Processing*, 2016, 64(13): 3444-3457.

[60] Heckbert P. Color image quantization for frame buffer display. In *Proc. the 9th Annual Conf. Computer Graphics and Interactive Techniques*, July 1982, pp.297-307.

[61] Mallows C. Another comment on o'cinneide. *The American Statistician*, 1991, 45(3): 257.

[62] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv: 1502.03167*, 2015. https://arxiv.org/abs/1502.03167, May 2017.

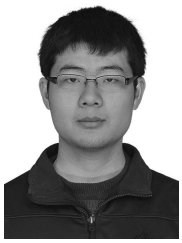
[63] Netzer Y, Wang T, Coates A, Bissacco A, Wu B, Ng A Y. Reading digits in natural images with unsupervised feature learning. In *Proc. Workshop on Deep Learning and Unsupervised Feature Learning*, Dec. 2011.

[64] Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z H, Karpathy A, Khosla A, Bernstein M, Berg A C, Li F F. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 2015, 115(3): 211-252.

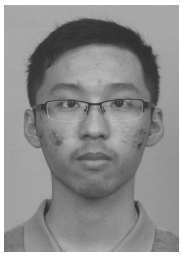
[65] Gysel P, Motamedi M, Ghiasi S. Hardware-oriented approximation of convolutional neural networks. *arXiv: 1604.03168*, 2016. http://arxiv.org/abs/1604.03168, May 2017.

[66] Taylor A, Marcus M, Santorini B. The Penn Treebank: An overview. In *Treebanks*, Abeillé A(ed.), Springer, 2003, pp.5-22.

Shu-Chang Zhou received his B.E. degree in electronic engineering and M.S. degree in computer science from Tsinghua University, Beijing, and National University of Singapore, Singapore, in 2004 and 2005 respectively. He had worked in Institute of Computing Technology, Chinese Academy of Sciences, Beijing, and Google Research before joining Megvii Inc. in 2014. His research interests include statistical machine learning and computer vision.



Yu-Zhi Wang received his B.S. degree in telecommunication engineering in 2012 from Xidian University, Xi'an. He is currently a Ph.D. candidate in the Department of Electronic Engineering in Tsinghua University, Beijing. He joined Megvii Inc. in 2016 as an intern. His research interests include wireless sensor networks, machine learning and deep neural networks.



He Wen is currently an undergraduate student in the Department of Computer Science and Technology in Tsinghua University, Beijing. He joined Megvii Inc. in 2015 as an intern.

Qin-Yao He is currently an undergraduate student in the Department of Computer Science and Technology in Tsinghua University, Beijing. He joined Megvii Inc. in 2015 as an intern.

Yu-Heng Zou is currently an undergraduate student in the School of Electronics Engineering and Computer Science, Peking University, Beijing. He joined Megvii Inc. in 2016 as an intern.

Appendix

A.1 Eliminating All Floating Point Operations During Inference

Recall that the i-th layer of a QNN is like:

$$\boldsymbol{X}_i^{\mathrm{q}} = Q_{\mathrm{A}}(\sigma_i(\boldsymbol{W}_i^{\mathrm{q}}\boldsymbol{X}_{i-1}^{\mathrm{q}} + \boldsymbol{b}_i)),$$
$$\boldsymbol{W}_i^{\mathrm{q}} = Q_{\mathrm{W}}(\boldsymbol{W}_i),$$

where σ_i is the activation function, and Q_{A} and Q_{W} are quantization functions of activations and weights respectively.

Below we assume the following conditions:

1) $\boldsymbol{W}_i^{\mathrm{q}}$ can be represented as fixed point numbers scaled by a floating point scalar α, i.e., $\boldsymbol{W}_i^{\mathrm{q}} = \alpha\boldsymbol{W}_{\mathrm{f}}$, where $\boldsymbol{W}_{\mathrm{f}}$ is fixed point numbers;

2) $\boldsymbol{X}_{i-1}^{\mathrm{q}}$ contains only fixed point numbers;

3) σ_i is a monotone function.

We next show that under these assumptions, the computation of $\boldsymbol{X}_i^{\mathrm{q}}$ can be done by operations between fixed point numbers.

First, by replacing the variables we have:

$$\boldsymbol{X}_i^{\mathrm{q}} = Q_{\mathrm{A}}(\sigma_i(\alpha\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}} + \boldsymbol{b}_i)).$$

As Q_{A} is a uniform quantization function, it can be computed by comparing values of $\sigma_i(\alpha\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}} + \boldsymbol{b}_i)$ with a sequence of thresholds $h_1, h_2, \cdots, h_n$. As σ_i is monotone, and w.l.o.g. assume $\alpha > 0$, the comparison can equivalently be done between $\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}}$ and

$$\frac{1}{\alpha}(\sigma_i^{-1}(h_1) - b),\ \frac{1}{\alpha}(\sigma_i^{-1}(h_2) - b),\ \cdots,\ \frac{1}{\alpha}(\sigma_i^{-1}(h_n) - b).$$

As $\boldsymbol{W}_{\mathrm{f}}$ and $\boldsymbol{X}_{i-1}^{\mathrm{q}}$ are fixed point numbers, their product is necessarily made of fixed point numbers; hence there exists a sufficiently large integer K such that $2^K\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}}$ are integers. The comparison required for computing Q_{A} can be done by comparing $2^K\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}}$ with integers

$$\lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_1) - b)\rfloor,\ \lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_2) - b)\rfloor,\ \cdots,$$
$$\lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_n) - b)\rfloor.$$

Hence the computation of $\boldsymbol{X}_i^{\mathrm{q}}$ can be done by the comparison between fixed point numbers $\boldsymbol{W}_{\mathrm{f}}\boldsymbol{X}_{i-1}^{\mathrm{q}}$ with the following thresholds that can be pre-computed and stored (hence eliminating the need for floating point operations during inference):

$$2^{-K}\lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_1) - b)\rfloor,\ 2^{-K}\lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_2) - b)\rfloor,$$
$$\cdots, 2^{-K}\lfloor\frac{2^K}{\alpha}(\sigma_i^{-1}(h_n) - b)\rfloor.$$

A.2 Training Algorithm of QNNs

For completeness we outline the training algorithm of QNNs in Algorithm A1. Weights, activations and gradients are quantized by quantization functions Q_{W}, Q_{A} and Q_{G}, respectively, which are applied to weights, activations and gradients respectively. C stands for the cost function of the neural network. *backward_input* and *backward_weight* are functions derived from the chain rule for computing gradients with respect to inputs and weights, respectively. The *Update* function is determined by the learning rule used. The algorithm extends Algorithm 1 of Hubara *et al.*[51] to include the quantization of gradients, and the multi-bit quantization.

Algorithm A1. Training an L-Layer CNN with W-Bit Weights and A-Bit Activations Using G-Bit Gradients

Require: a minibatch of inputs and labels $(\boldsymbol{X}_0, \boldsymbol{Y})$, previous weights $\boldsymbol{W}$, learning rate η
Ensure: updated weights $\boldsymbol{W}^{t+1}$

```
{1. Computing the parameter gradients:}
{1.1 Forward Propagation:}
for i = 1 → L do
    W_i^q ← Q_W(W_i)
    X̃_i ← X_{i-1}^q W_i^q + b_i
    X_i ← σ(X̃_i)
    if k < L then
        X_i^q ← Q_A(X_i)
    end
    Optionally apply pooling
end
{1.2 Backward propagation:}
Compute g_L = ∂C/∂X_L knowing X_L and label Y
for i = L → 1 do
    Back-propagate g_i through activation function σ
    g_i^q ← Q_G(g_i)
    g_{i-1} ← backward_input(g_i^q, W_i^q)
    g_{W_i} ← backward_weight(g_i^q, X_{i-1}^q)
    Back-propagate gradients through pooling layer if
    there is one
end
{2. Accumulating the parameters gradients:}
for k = 1 → L do
    g_i = g_i^q ∂W_i^q/∂W_i
    W_i^{t+1} ← Update(W_i, g_i, η)
end
```

A.3 Proof of Proposition 1

Proof. The step after histogram equalization in Algorithm 1 maps the following half open (close) intervals into quantization values:

$$[0,\ \frac{1}{2^K}),\ [\frac{1}{2^K},\ \frac{2}{2^K}),\ \cdots,\ [\frac{2^K-1}{2^K},\ 1].$$

Hence it is sufficient to prove the counting statements for these intervals after the application of Algorithm 2.

First of all, as each call of `HistogramEqualize` either produces two recursive calls or terminates depending on *level* variable, the call relation of any invocation of `HistogramEqualize` will form a balanced binary tree. For clarity, we note as $\boldsymbol{M}_k^l$, $\boldsymbol{M}_k^g$ the corresponding $\boldsymbol{M}^l$, $\boldsymbol{M}^g$, respectively, used for a depth k node of the binary tree.

By the assumption of $\boldsymbol{M}^l$, $\boldsymbol{M}^g$, we have $\frac{1}{\gamma} \leqslant \frac{\sum \boldsymbol{M}_k^l}{\sum \boldsymbol{M}_k^g} \leqslant \gamma$. At the leaf nodes, the application will be at most K times; hence the number of entries in leaf nodes will be different by at most γ^{2k} times.

What remains to be proved is that no two leaf nodes produce the same quantized value. We create an auxiliary variable $D_n^k \in \{0, 1\}$ to record whether a depth k node is on the right branch of their depth $k-1$ parent. We can prove that node n will map values to the interval $\sum_k D_n^k 2^{k-1}$, by observing that at line 1 of Algorithm 2, $\dfrac{1}{2}$ will only be added if the right branch of the call tree is taken.

As $\sum_k D_n^k 2^{k-1}$ is unique for all nodes, we complete the proof. □

A.4 Quantization of GRU

We first investigate the quantization of GRU as it is structurally simpler. The basic structure of GRU cell may be described as follows:

$$\begin{aligned}
\boldsymbol{z}_t &= \sigma(\boldsymbol{W}_z \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\boldsymbol{r}_t &= \sigma(\boldsymbol{W}_r \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\widetilde{\boldsymbol{h}_t} &= \tanh(\boldsymbol{W} \cdot [\boldsymbol{r}_t \circ \boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\boldsymbol{h}_t &= (\boldsymbol{1} - \boldsymbol{z}_t) \circ \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \circ \widetilde{\boldsymbol{h}_t},
\end{aligned}$$

where $\boldsymbol{1}$ is a vector with all entries being 1, σ stands for the sigmoid function, "$\cdot$" stands for the dot product, $[\boldsymbol{x}, \boldsymbol{y}]$ stands for the concatenation of two vectors $\boldsymbol{x}$ and $\boldsymbol{y}$, and $\circ$ stands for the Hadamard product.

Recall that to benefit from the speed advantage of bit convolution kernels, we need to multiply the two matrix inputs in low-bit forms, such that the dot product can be calculated by bitwise operation. For plain feed forward neural networks, as the convolutions take up most of computation time, we can get decent acceleration by the quantization of inputs of convolutions and their weights. But when it comes to more complex structures like GRU, we need to check the bitwidth of each interlink.

Except for matrix multiplications needed to compute $\boldsymbol{z}_t$, $\boldsymbol{r}_t$ and $\widetilde{\boldsymbol{h}_t}$, the gate structures of $\widetilde{\boldsymbol{h}_t}$ and $\boldsymbol{h}_t$ bring in the need for element-wise multiplication. As the outputs of the sigmoid function may have higher bitwidths, the element-wise multiplication may need to be done between floating point numbers (or in higher bitwidth format). As $\widetilde{\boldsymbol{h}_t}$ and $\boldsymbol{h}_t$ are also the inputs to computations at the next timestamp, and noting that a quantized value multiplied by a quantized value will have a larger bitwidth, we need to insert additional quantization steps after element-wise multiplications.

Another problem with the quantization of GRU structure is that the value ranges of gates are different. The range of tanh is $[-1, 1]$, which is different from the value range $[0, 1]$ of $\boldsymbol{z}_t$ and $\boldsymbol{r}_t$. If we want to preserve the original activation functions, we will have the following quantization scheme:

$$\begin{aligned}
\boldsymbol{z}_t &= \sigma(\boldsymbol{W}_z \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\boldsymbol{r}_t &= \sigma(\boldsymbol{W}_r \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\widetilde{\boldsymbol{h}_t} &= \tanh(\boldsymbol{W} \cdot [2Q_k(\frac{1}{2}(\boldsymbol{r}_t \circ \boldsymbol{h}_{t-1}) + \frac{1}{2}) - 1, \boldsymbol{x}_t]),\\
\boldsymbol{h}_t &= 2Q_k(\frac{1}{2}((\boldsymbol{1} - \boldsymbol{z}_t) \circ \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \circ \widetilde{\boldsymbol{h}_t}) + \frac{1}{2}) - \boldsymbol{1},
\end{aligned}$$

where we assume the weights $\boldsymbol{W}_z, \boldsymbol{W}_r, \boldsymbol{W}$ have already been quantized to the closed interval $[-1, 1]$, and input $\boldsymbol{x}_t$ has already been quantized to $[-1, 1]$.

However, we note that the quantization function already has an affine transform to shift the value range. To simplify the implementation, we replace the activation functions of $\widetilde{\boldsymbol{h}_t}$ to be the sigmoid function, so that $(\boldsymbol{1} - \boldsymbol{z}_t) \circ \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \circ \widetilde{\boldsymbol{h}_t} \in [0, 1]$.

Summarizing the above considerations, the quantized version of GRU could be written as

$$\begin{aligned}
\boldsymbol{z}_t &= \sigma(\boldsymbol{W}_z \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\boldsymbol{r}_t &= \sigma(\boldsymbol{W}_r \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]),\\
\widetilde{\boldsymbol{h}_t} &= \sigma(\boldsymbol{W} \cdot [Q_k(\boldsymbol{r}_t \circ \boldsymbol{h}_{t-1}), \boldsymbol{x}_t]),\\
\boldsymbol{h}_t &= Q_k((\boldsymbol{1} - \boldsymbol{z}_t) \circ \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \circ \widetilde{\boldsymbol{h}_t}),
\end{aligned}$$

where we assume the weights $\boldsymbol{W}_z, \boldsymbol{W}_r, \boldsymbol{W}$ have already been quantized to $[-1, 1]$, and input $\boldsymbol{x}_t$ has already been quantized to $[0, 1]$.

A.5 Quantization of LSTM

The structure of LSTM can be described as follows:

$$\begin{aligned}
\boldsymbol{f}_t &= \sigma(\boldsymbol{W}_{\mathrm{f}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{f}}),\\
\boldsymbol{i}_t &= \sigma(\boldsymbol{W}_{\mathrm{i}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{i}}),\\
\widetilde{\boldsymbol{C}_t} &= \tanh(\boldsymbol{W}_{\mathrm{C}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{i}}),
\end{aligned}$$

$$\boldsymbol{C}_t = \boldsymbol{f}_t \circ \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \circ \widetilde{\boldsymbol{C}_t},$$
$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_{\mathrm{o}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{o}}),$$
$$\boldsymbol{h}_t = \boldsymbol{o}_t \circ \tanh(\boldsymbol{C}_t).$$

Different from GRU, $\boldsymbol{C}_t$ cannot be easily quantized, since the value has not been bounded by activation functions like tanh. This difficulty comes from structure design and cannot be alleviated without introducing extra facility to clip value ranges. But it can be noted that the computations involving $\boldsymbol{C}_t$ are all element-wise multiplications and additions, which may take much less time than computing matrix products. For this reason, we leave $\boldsymbol{C}_t$ to be floating point numbers.

To simplify implementation, tanh activation for output may be changed to the sigmoid function.

Summarizing above changes, the formula for quantized LSTM can be:

$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}_{\mathrm{f}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{f}}),$$
$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_{\mathrm{i}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{i}}),$$
$$\widetilde{\boldsymbol{C}_t} = \tanh(\boldsymbol{W}_{\mathrm{C}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{i}}),$$
$$\boldsymbol{C}_t = \boldsymbol{f}_t \circ \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \circ \widetilde{\boldsymbol{C}_t},$$
$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_{\mathrm{o}} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_{\mathrm{o}}),$$
$$\boldsymbol{h}_t = Q_{\mathrm{k}}(\boldsymbol{o}_t \circ \sigma(\boldsymbol{C}_t)),$$

where we assume the weights $\boldsymbol{W}_{\mathrm{f}}, \boldsymbol{W}_{\mathrm{i}}, \boldsymbol{W}_{\mathrm{C}}, \boldsymbol{W}_{\mathrm{o}}$ have already been quantized to $[-1, 1]$, and input $\boldsymbol{x}_t$ have already been quantized to $[0, 1]$.